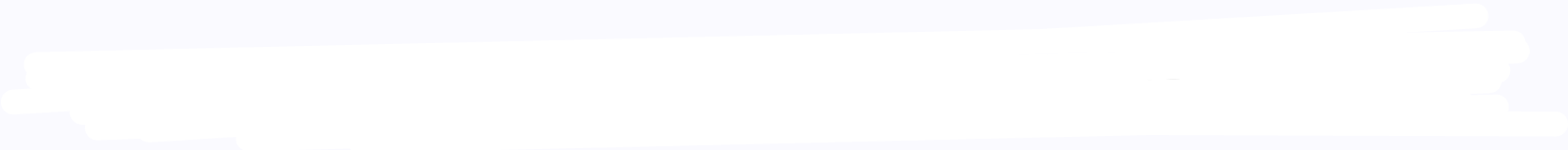


Problème d'allocation de registres

A thick, horizontal yellow brushstroke underline is positioned below the title text.

Qu'est-ce qu'un registre ?

Un registre est un espace de stockage.

Hiérarchie de la mémoire :

- Registre
- Cache processeur (L1,L2,L3,L4)
- Mémoire vive
- Disque dur

La compilation

Front-end :

- L'analyse lexicale
- Le parsing (analyse syntaxique)
- L'analyse statique du code

Middle-end :

- L'optimisation

Back-end :

- La génération du code

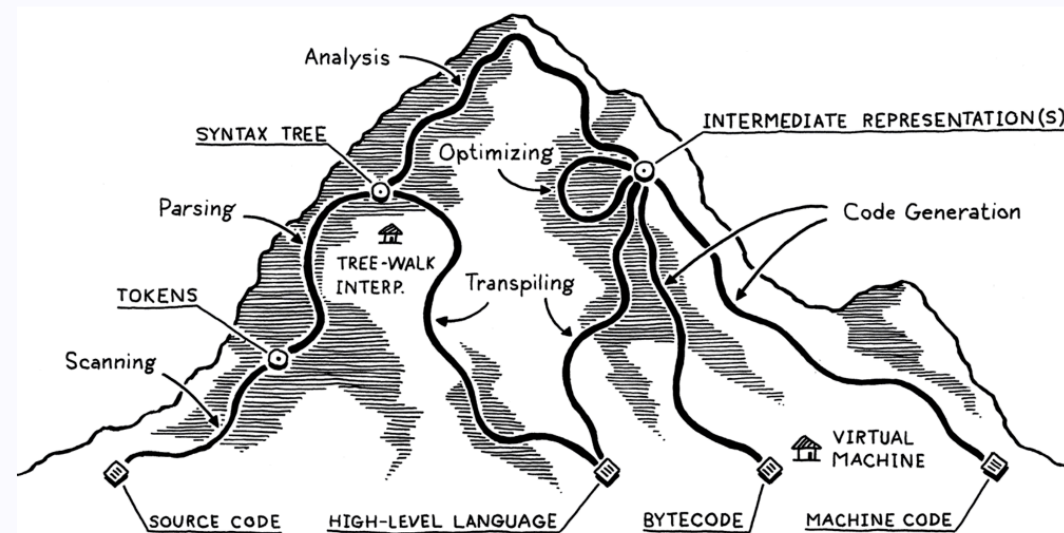


Image de "Crafting Interpreters"

Allocation naïve :



Pile

r1

r2

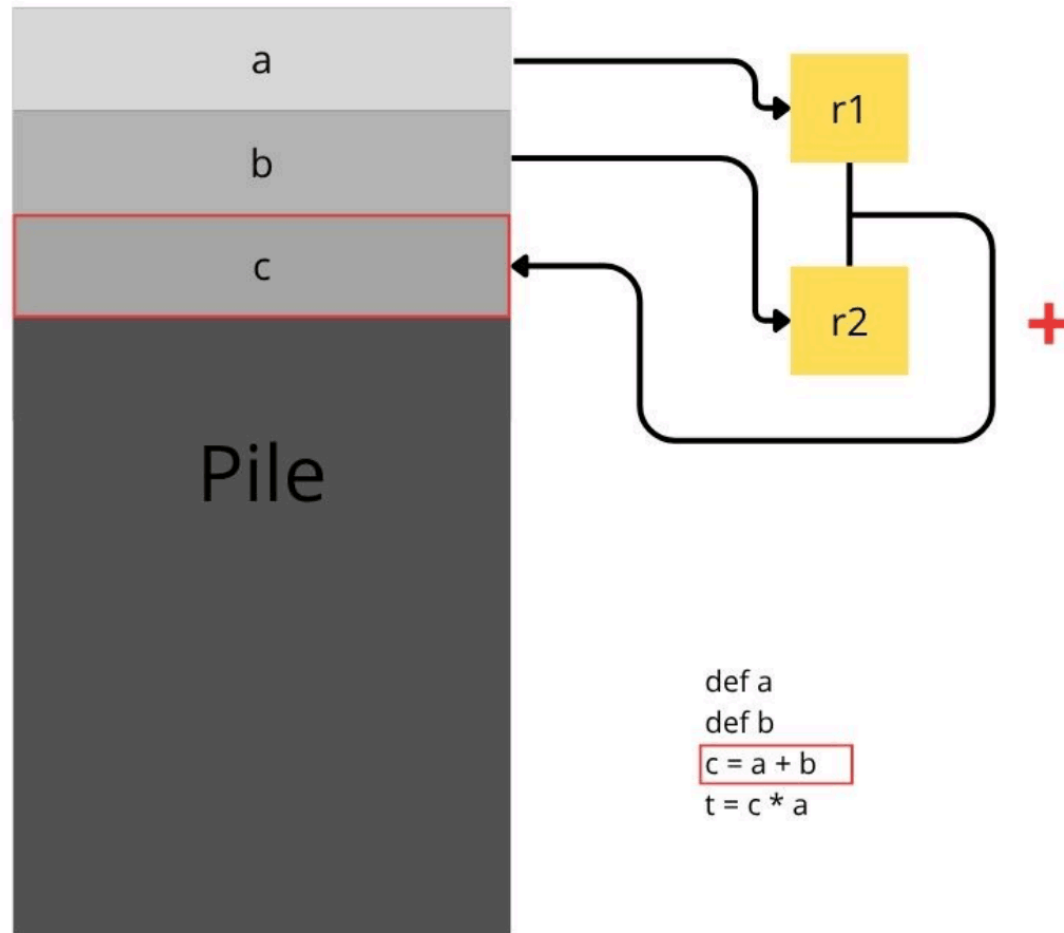
```
def a
def b
c = a + b
t = c * a
```

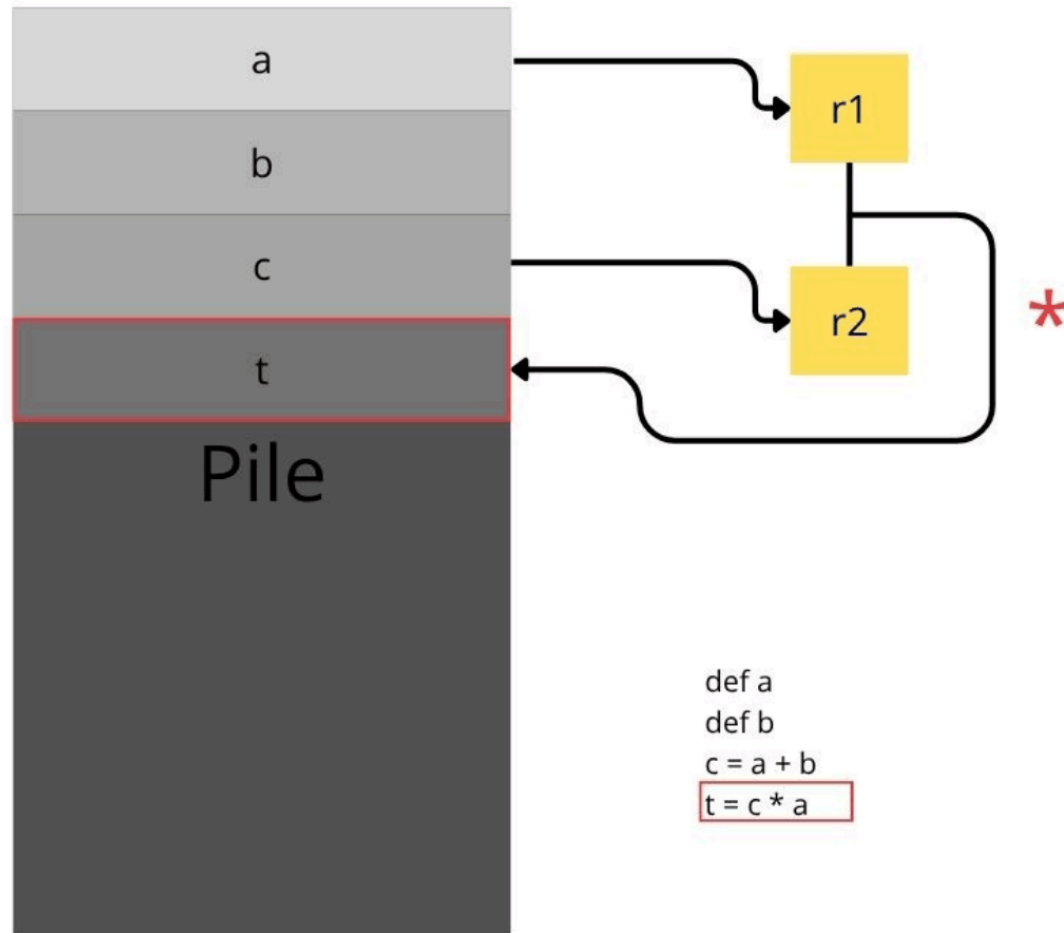


r1

r2

```
def a
def b
c = a + b
t = c * a
```

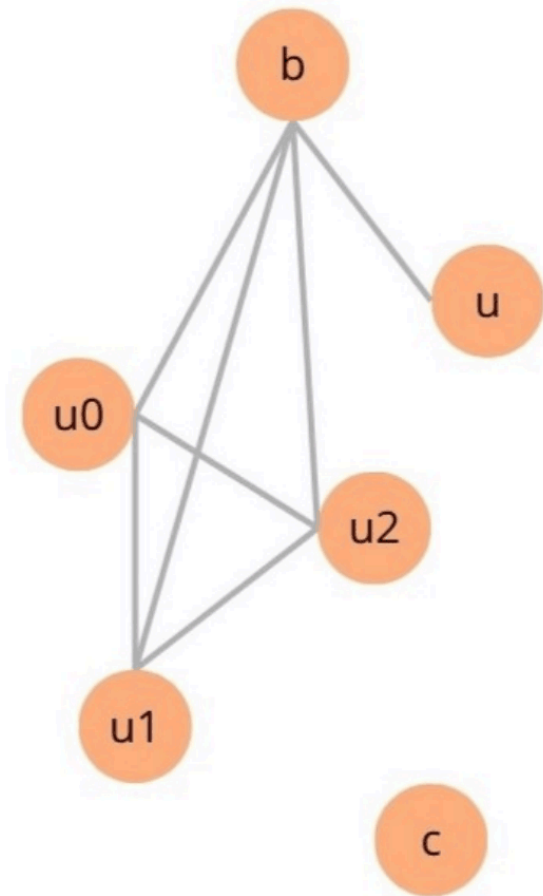




Exemple de graphe d'interférence :

```
int main(void){  
    int b = 2;  
    int u0 = 1;  
    int u1 = 0;  
    int u2 = 1;  
    int u = u0 + u1*b + u2*b*b;  
    int c = b;  
  
    return 0;  
}
```

Graphe d'interférence



Coloration de graphe :

- n variables = n noeuds
- Variables vivant en même temps dans le programme = Noeuds voisins entre eux.
- K registres disponibles = K couleurs

Coloration de graphe -> problème NP

Problèmes de classe NP

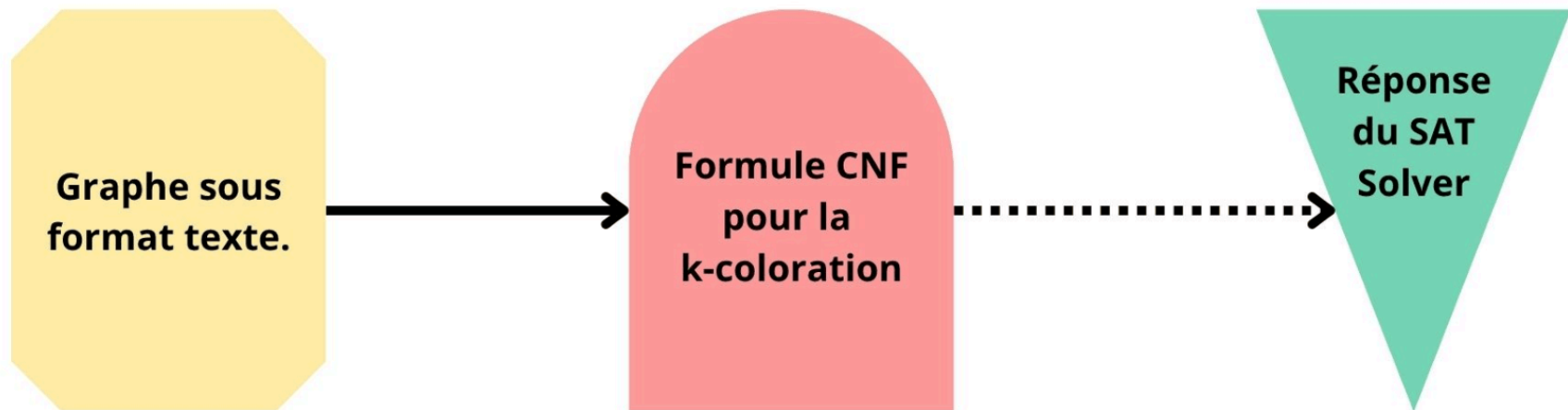
- Certificat de taille polynomiale en celle de l'instance
- Vérificateur en temps polynomial en la taille de l'instance

Problème SAT

Problème SAT (satisfiabilité booléenne) -> NP complet

Donc réduction de la coloration de graphe à une formule propositionnelle possible.

Mon algorithme :



Mon SAT solveur :

Entrée : Formule CNF

Sortie :

- La colorabilité du graphe
- Une valuation

Un *parcours de la valuation* nous donne la *couleur* des noeuds.

Fonctionnement :

Etape 1 : Recherche de clauses 'unitaires'.

Etape 2 : Recherche de littéraux 'purs'.

Etape 3 : Choix d'un littéral de branchement par une heuristique.

Après l'exécution de chaque étape, on retourne à l'étape 1.

Heuristique

On compte les occurrences positives et négatives des littéraux.

On choisit le littéral associé à $\max(n_{\max}^-, n_{\max}^+)$ et on lui attribue la valuation associée.

Types

```
typedef int litteral;

struct cnf {
    litteral **clauses; // tableau des clauses disjonctives
    int nb_clauses;
    int nb_variables;
};

typedef struct cnf cnf;

struct litt_occurrences {
    int pos;
    int neg;
};

typedef struct litt_occurrences litt_occurrences;
```

Implémentation

```
litteral get_decision_litteral(cnf *f, litt_occurrences *occs){
    int n_plus = 0;
    litteral lit_plus = 0;
    int n_moins = 0;
    litteral lit_moins = 0;
    for(int lit = 1; lit<=f->nb_variables; lit++){
        if(occs[lit].pos > n_plus){
            n_plus = occs[lit].pos;
            lit_plus = lit;
        }
        if(occs[lit].neg > n_moins){
            n_moins = occs[lit].neg;
            lit_moins = lit;
        }
    }
    if(n_plus>n_moins) return lit_plus;
    return -lit_moins;
}
```

Cas de fin :

- La cnf vaut vrai, elle est donc satisfiable par la valuation créée
- Une clause de la cnf vaut faux, la valeur d'une des variables de branchements doit être modifiée.

Performances

Pour $k=4$, sur le graphe d'interférence précédent :

Mon sat solveur :

Presque instantané.

Minisat :

Presque instantané.

Performances

Pour $k=5$, sur un graphe a 30 sommets et 62 arêtes :

Mon sat solveur :

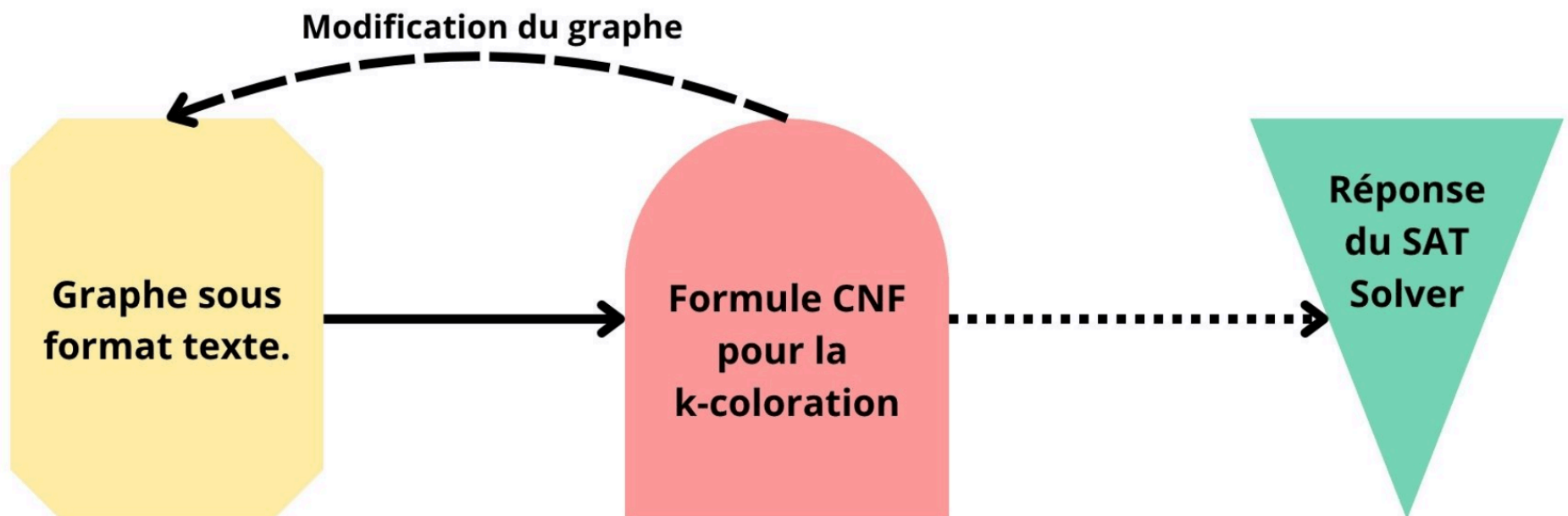
60 ms (en moyenne sur 20 essais).

Minisat :

2,75 ms (en moyenne sur 20 essais).

Mais, si manque de registre, alors coloration telle
quelle inutile.

D'où modification à apporter :

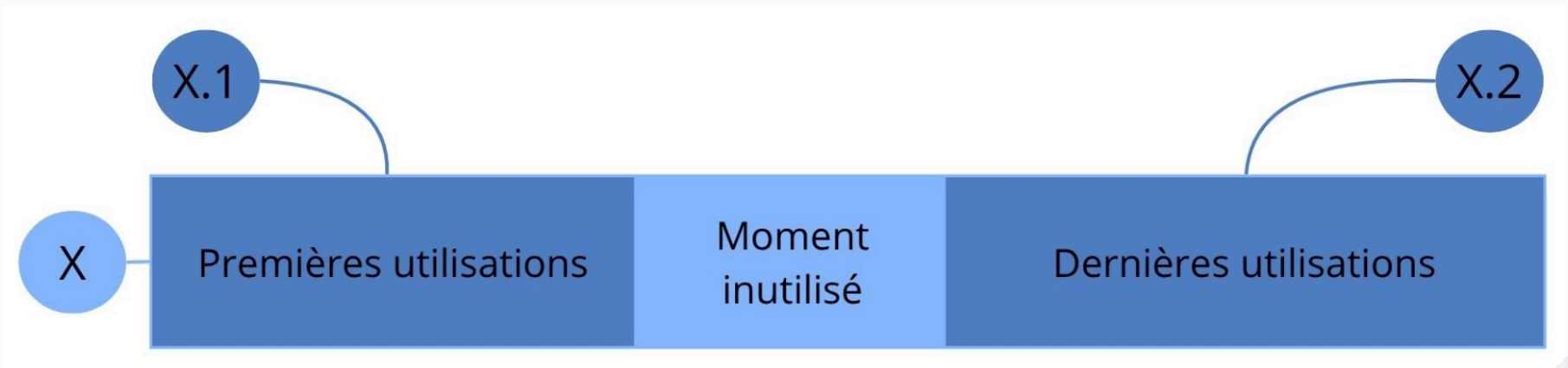


Etape de division :

(Modification du graphe)

1/ Etude des vies de nos variables -> sélection d'un noeud

2/ Séparation de ce noeud en deux neuds différents



Mais comment choisir quel noeud diviser ?

Un calcul de priorité de division possible :

p = 'priorité'

$u1$ = 'utilisation(s) et définition(s) hors d'une boucle'

$u2$ = 'utilisation(s) et définition(s) dans une boucle'

d = 'degrés'

$$p = \frac{u1 + 10 * u2}{d}$$

Merci pour votre attention

Sauf indication contraire, tout schéma de cette présentation a été fait sur Canva.